

Training Through the Inferno: An Inspectable OSRS Challenge Environment for RL

Valtteri Valo

27th May 2025

Abstract

The Old School RuneScape Inferno is an hour-scale solo game task built for expert human players with years of game experience. A run spans 69 waves. The first 66 waves have fixed enemy compositions but randomized placements, so each run asks the player to solve a fresh spatial puzzle of where to stand, what to block with pillars, which target to kill first, and when to switch prayers or spend supplies while several threats act at once. The final waves turn into a boss sequence whose mechanics differ from the hour of content before it, ending in TzKal-Zuk, a last-wave encounter with its own movement, shield, healer, and target-priority rules. We present a C simulator and PufferLib 4 training environment for this task. The environment exposes 744 raw simulator-state features, a 9-head action space with 89 total discrete choices, strict action masks, replayable failures, behavior logs, and a renderer tied to the same state used for training. A recurrent PufferNet policy trained for 160.5M environment steps on a single 4090 GPU reaches a 39.8% logged completion rate on full Inferno runs in under 13 minutes of training. In boss-focused training, it reaches win rates of over 95% in less than two minutes of training. The result shows that high-throughput recurrent RL can iterate on a long-horizon, expert-level game task and keep failures readable to domain experts.

1 Introduction and Contribution

The Old School RuneScape Inferno is a solo PvM challenge released in 2017, at the time the most challenging content in the game by far and still intimidating today. For a reader outside OSRS, the Inferno is a long sequence of tactical puzzles under mechanical pressure. Each wave has a fixed enemy composition, but enemy placements vary between runs. A small spawn change can turn the same wave from a simple prayer-switching problem into a positioning puzzle around pillars, line of sight, target priority, and supply risk. The challenge gets harsher as it goes, and later waves combine more threats and punish earlier resource mistakes. After wave 66, the task changes shape into a boss sequence, ending with TzKal-Zuk, whose shield movement, healer control, target priority, and multi-tasking pressure play nothing like the waves before it.

This paper is about environment construction. We do not wrap a game binary and report a leaderboard number. We build a simulator whose state, actions, masks, renderer, replay path, and logs form one contract. The same game facts used for training must be available when explaining watched behavior. If a policy looks wrong in replay, the logs and simulator should explain why, as plain domain expertise does not catch everything.

The practical question is whether a difficult player-readable game task can train fast enough for iteration while leaving policy failures visible to a human reviewer. The Inferno environment treats systems work, simulator fidelity, logs, and rendered replay as one because behavior bugs move across all four.

This places the environment near other game-derived RL tasks that use fast simulation to expose hard generalization or long-horizon behavior. Procgen uses procedurally generated games to measure sample efficiency and generalization, while NetHack combines fast simulation with sparse reward, stochasticity, exploration, and long-term planning [5, 6].

2 Environment Surface

The Inferno environment uses simulator-state observations instead of pixels. The current surface has 744 raw state features and a 9-head action space with 89 total discrete choices. The rollout row appends one mask entry per choice, for 833 floats total. One environment step is one OSRS game tick, or 0.6 seconds, simulated as instantaneous. Episodes can run for thousands of ticks. The native environment writes fixed-size observations and masks into PufferLib rollout buffers for thousands of parallel agents.

The observation vector contains player health, prayer, supplies, wave phase, gear state, combat stats, target state, weapon range, NPC slots, pillar health, pending hits, pending Zuk healer area attacks, and short movement forecasts.



Figure 1: An Inferno wave 69 replay frame exposes the training contract visually, from tick timing and target labels through arena state and a player-familiar client UI.

Positions are normalized and mostly egocentric or arena-relative. Categories use one-hot structure or slot membership. The design rule is to expose player-relevant state, then log whether the policy acts on it.

The action space is a structured interface over OSRS intent, with movement, overhead prayer, target selection, gear switching, food, potion, spell, special attack, and offensive prayer. Targets point to observation slots, which keeps the action interface stable when internal entity ids change. The mask writer validates every action head and records zero-valid-head failures. The model should decide whether to run west, pray magic, tag a healer, cast barrage, or drink a restore. Invalid actions should disappear before they reach the policy, so model capacity goes into the challenge itself.

Component	Design choice	Reason
Observation	744 raw state features	Exposes game state
Action	9 structured heads	Matches game intent
Masks	One entry per choice	Removes invalid client actions
Debug	Renderer, replay, logs	Makes failures inspectable
Training	4096 parallel agents	Keeps long episodes cheap

Table 1: The environment keeps training and debug facts in one contract.

Reward and score are separate. Reward can change during development. Score is intended to survive those edits. In this environment, reward is assigned through damage, healer tags, shield preservation, late-wave pressure, and final-phase priorities. Penalties, or negative rewards are not assigned. The training objective score in Table 3 is also separate from the archive `progress_score` used for Zuk-state selection. For full-start runs, it is win rate plus half-weighted wave progress on losses: $\text{winrate} + (1 - \text{winrate}) \cdot \text{mean_wave}/69 \cdot 0.5$. This explains why a 0.398293 win rate and 66.343330 mean wave produce a 0.687563 score.

3 Training and Iteration Loop

PufferLib was introduced as a way to make RL libraries and environments interoperate [3]. PufferLib 4 makes this loop practical because the bottleneck is the full edit-train-watch cycle. Puffer includes a native backend with CUDA C kernels, static memory, CUDA graphs, async environment workers, fused operations, action masks, PufferNet policies, and Protein hyperparameter sweeps [4]. The systems pressure is familiar in high-throughput RL. Sample Factory targets single-machine asynchronous RL, and EnvPool treats parallel environment execution as a first-class bottleneck [7, 8].

The recurrent model is PufferNet, which combines MinGRU with highway-style output mixing [9, 10]. MinGRU supports parallel-scan training across sequences and cheap recurrent inference during rollouts. This fits game tasks where the policy needs memory during rollout, but training must still use long enough sequences to learn delayed consequences.

The current Inferno config uses 4096 parallel agents, two rollout buffers, 16 worker threads, a two-layer 512 hidden PufferNet, a 16 tick horizon, priority replay, and a replay ratio of 4. The horizon covers 9.6 seconds of game time. Protein sweeps search over learning rate, horizon, discounting, advantage settings, replay, vector size, model width, curriculum fractions, and reward weights. Protein models both score and wall-clock cost, which matters because a high score from a slow or unstable setting may be a poor tradeoff and development tool.

The central engineering lesson is that the simulator and learner cannot be debugged separately. Training ruthlessly exposes flawed observations and reward traps. Renderer checks expose bugs for instance in projectile anchoring, pathing, monster prioritization, and general decision making. Human controllability exposes when the action interface is not playable or something "feels off" to a domain expert. Some fixes cross every surface. Prayer flick semantics [2] touch environment logic, GUI paths, human input paths, render paths, trainer bindings, and tests. The useful loop is often to train, watch, find a player-readable failure, trace the simulator, add a contract test, and sweep again. Although tongue-in-cheek, a hyperparameter sweep is considered one unit of compute in the Puffer paradigm.

Observed failure	Visible signal	Contract added
Wrong prayer timing	Prayer correctness and death context	Pending-hit observations
Stale target options	Zero-valid-head counters	Strict target masks
Missed Zuk healers	Healer tag and kill logs	Healer phase diagnostics
Broken Zuk movement	Replay and human click pathing	Shield-path checks
Wrong projectile source	Rendered flight path	Anchored projectile endpoints

Table 2: Watching policies explains behavior bugs and odd decisions. Metrics are necessary to catch more aggregate failures.

4 Current Best Run

The current best run in Table 3 comes from the latest current-compatible sweep artifact. The trial used exact step-out forecast observations, priority replay, 4096 parallel agents, a two-layer 512 hidden PufferNet, and 160.5M environment steps.

Metric	Value
Training steps	160.5M
Logged win rate	0.398293
Training objective score	0.687563
Average wave reached, out of 69	66.343330
Average lowest Zuk HP, lower is better	323.682892
Correct prayer rate	0.871236
Runs clearing all Zuk healers	0.438476
Final training throughput	about 208k steps/s

Table 3: Stored development telemetry for the current-compatible **xbeeukiz** Inferno checkpoint.

Read these numbers as a training snapshot. The environment, action surface, and visual contract keep changing during development. The result shows logged full clears and frequent late-Zuk states in the current environment.

These numbers do not prove the task is solved, because the task definition is still moving. Future simulator versions need fresh evaluation, and replays still need domain expertise checks for simulator-specific behavior. The longest standing stable metric is the eval score, for good reasons.

In an inspectable game task, success cannot be only a scalar. A policy that completes the objective by leaning on a simulator bug has not solved the task in the player-facing sense. The environment must keep these failures visible.

A stricter public evaluation release should freeze the observation and action spaces, pin the checkpoint metadata, replay sampled wins and losses, and report behavioral counters alongside score. A policy should be judged on full clears and stability in achieving them.

5 Lessons and Limitations

First, game environments need a player-facing contract. If policy behavior looks wrong in replay, the logs and simulator should explain why. This requirement is functional. In the Inferno, rendering is used to catch bugs not visible in scalar training graphs.

Second, strict action masks are part of the task definition. OSRS has many impossible or detrimental actions like casting unavailable spells, switching to already equipped gear, or stepping into blocked tiles. Masking these actions removes interface noise that human players would never engage in so the model can spend capacity on game decisions.

Third, reward is not score. Reward is an engineering tool for credit assignment. Score is the stable development measure. Keeping them separate makes the project less brittle while reward weights and curricula change.

Fourth, higher throughput makes the development loop more empirical. PufferLib 4’s systems choices are practical. The same lesson appears across Sample Factory and EnvPool: throughput changes what researchers can afford to measure, debug, and repeat [7, 8]. Puffer makes it cheap enough to find out whether a simulator fix or observation change helps.

Fidelity is the main limitation. The environment simulates the encounter mechanics used for training and uses cache-derived assets for visual inspection. Recreating private game logic remains slow work, even for domain experts. Expert knowledge makes failures interpretable, but it can also bias training. The safest path is to keep tests, render checks, replay checks, and human play close to the training surface. Relying on the smallest possible set of scaffolding is central.

The OSRS Inferno environment suggests an evaluation pattern for long-horizon game RL: player-readable state, structured actions, strict masks, replayable failures, and high-throughput recurrent training over long episodes.

References

- [1] OSRS Wiki. Inferno. <https://oldschool.runescape.wiki/w/Inferno>
- [2] OSRS Wiki. Prayer flicking. https://oldschool.runescape.wiki/w/Prayer#Prayer_flicking
- [3] Joseph Suarez. PufferLib: Making Reinforcement Learning Libraries and Environments Play Nice. arXiv:2406.12905, 2024. <https://arxiv.org/abs/2406.12905>
- [4] PufferAI. PufferLib docs. <https://puffer.ai/docs.html>
- [5] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging Procedural Generation to Benchmark Reinforcement Learning. arXiv:1912.01588, 2019. <https://arxiv.org/abs/1912.01588>
- [6] Heinrich Kuttler, Nantas Nardelli, Alexander H. Miller, Roberta Raileanu, Marco Selvatici, Edward Grefenstette, and Tim Rocktaschel. The NetHack Learning Environment. arXiv:2006.13760, 2020. <https://arxiv.org/abs/2006.13760>
- [7] Aleksei Petrenko, Zhehui Huang, Tushar Kumar, Gaurav Sukhatme, and Vladlen Koltun. Sample Factory: Egocentric 3D Control from Pixels at 100000 FPS with Asynchronous Reinforcement Learning. arXiv:2006.11751, 2020. <https://arxiv.org/abs/2006.11751>
- [8] Jiayi Weng, Min Lin, Shengyi Huang, Bo Liu, Denys Makoviichuk, Viktor Makoviychuk, Zichen Liu, Yufan Song, Ting Luo, Yukun Jiang, Zhongwen Xu, and Shuicheng Yan. EnvPool: A Highly Parallel Reinforcement Learning Environment Execution Engine. arXiv:2206.10558, 2022. <https://arxiv.org/abs/2206.10558>
- [9] Leo Feng, Frederick Tung, Mohamed Osama Ahmed, Yoshua Bengio, and Hossein Hajimirsadeghi. Were RNNs All We Needed? arXiv:2410.01201, 2024. <https://arxiv.org/abs/2410.01201>
- [10] Rupesh Kumar Srivastava, Klaus Greff, and Jurgen Schmidhuber. Highway Networks. arXiv:1505.00387, 2015. <https://arxiv.org/abs/1505.00387>